CS 344: Algorithms **Rutgers: Spring 2026**

Homework #3

Name(s): Iris Martinez

Homework Policy

- You may consult all course materials (lecture notes, textbook, slides, etc.) while writing your solutions. No external resources are permitted.
- **All submissions should be typeset in LaTeX, handwritten solutions are not accepted**.
- Remember to always **prove the correctness** of your algorithms and **analyze their running time** (or any other efficiency measure asked in the question).
- Homework assignments are completed and submitted in fixed groups of 3 students. Only **one member of the group submits** the full solution on Canvas. The submission must include the names of all group members. The other two members do **not need to submit anything separately**.
- You may discuss the problems with any of your classmates, even those outside your group. However, **each group must write and submit their solutions independently**.
- Attempt the challenge problems only if you find them interesting.

Acknowledgment (to be reviewed and edited as needed)

I, **Iris Martinez**, worked on this assignment individually.**I hereby affirm that all external assistance utilized in the preparation of this document has been properly acknowledged**.

Problem 1. You are given an array of characters $S[1:n]$ such that each $S[i]$ is a small case letter from the English alphabet. You are also provided with a black-box algorithm *dict* that given two indices $i, j \in [n]$, $dict(i, j)$ returns whether the sub-array $S[i:j]$ in array S forms a valid word in English or not in $O(1)$ time. (Note that this algorithm is provided to you and you do not need to implement it in any way).

Design and analyze a dynamic programming algorithm to find whether the given array S can be partitioned into a sequence of valid words in English. The runtime of your algorithm should be $O(n^2)$.

Example: Input Array: $S = [maythe force bewithyou]$.

Assuming the algorithm *dict* returns that $may, the, force, be, with$ and you are valid words (this means that for instance, for may we have $dict(1, 3) =$ True), this array can be partitioned into a sequence of valid words.

Solution.

Algorithm:

We set up a memo array of length n+1. The last value is true and represents an empty string that is splittable into zero valid words.

Now we do this:

```
for i = n to zero:
  for j up to i:
    if dict(i,j) == true and (table[j+1] ==true)
      table[i] = true
```

These loops check if there is a valid split of the last n-i characters.

The result will be in table[0].

Proof:

- **Base Case:** For an empty string, it must be splittable into zero valid words.
- **Inductive Step:** For a string from i to n, it is splittable if there is a value j where the string from i to j is a valid word and the string after j to n is splittable.

  In the most trivial case. A string of length one (like the letter 'a') is splittable because the substring from 0 to 0 is a word 'a' and the substring after ("") is splittable.

  As we step that boundary down further and further, we will keep finding substrings of the larger array that can be perfectly partitioned into valid words. If this propagates down to the first character, then we know the whole string is splittable. The table in this case saves us the work of having to recalculate if every substring to the right of the boundary is splittable, instead giving a memo of what substrings were splittable.

Runtime: The main work of this occurs in two loops over the whole of the string, giving $O(n^2)$ complexity

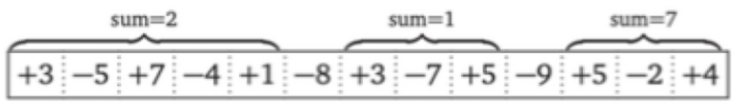
---

Problem 2. Suppose you are given an array $A[1:n]$ of integers that can be positive or negative. A *contiguous* subarray $A[i:j]$ of A is called a **positive** subarray if sum of its entries is strictly positive, i.e.,

$$\sum_{k=i}^{j} A[k] > 0.$$

Design and analyze a dynamic programming algorithm that in $O(n^2)$ time finds the smallest number of positive contiguous subarrays of A such that every positive entry of A belongs to exactly one of these subarrays. You can assume that A has at least one positive entry and thus the answer is *at least* one.

Example: The correct answer for the following array is 3 subarrays.



Solution.

```
def function(A[n])
  memo = [0:n]
  memo[0] = 0
  memo[1:n] = +infty
  for i in range 1:n:
    if A[i] <= 0:
      memo[i] = memo[i-1]

    cur = 0
    for j in (i down to 1):
      cur = cur + A[j]
      if cur > 0 and memo[i-1] != +infty:
        memo[i] = min(memo[i], memo[i-1] + 1)
  return memo[n]
```

1. **Base:** An empty array requires 0 positive contiguous subarrays.
2. **Inductive Step:**

   If we assume that the memo array stores correct solutions for the array from 0 to an index i, then this is what happens when we calculate memo[i+1]:

   If A[i+1] is negative, it doesn't affect our solution and we can carry over the answer from a[i]. So memo[i+1] = memo[i]

   By taking the running sum from the index down to 0, we determine that the subsequent indices are part of the same subarray and therefore have the same memoized solution. Only when the current index we're adding makes the solution smaller does it change.

   So either the new value forms part of one of the existing subarrays and the previous value gets carried forward, or it doesn't and we add one to the solution at that index.
3.

This has two nested loops that iterate over the array, so its $O(N^2)$

Problem 3. We define a number to be **special** if it can be written as:

$$n = a \cdot 197 + b \cdot 232$$

where a and b are non-negative integers. For example:

- 696 is special because it can be written as $696 = 0 \cdot 197 + 3 \cdot 232$.
- 2412 is special because it can be written as $2412 = 4 \cdot 197 + 7 \cdot 232$.
- 267 is not special (since it cannot be written as $a \cdot 197 + b \cdot 232$ for non-negative a and b).

The goal of this problem is to write a dynamic programming algorithm **Special(n)** that, given a positive integer n, either finds non-negative integers a and b such that $n = a \cdot 197 + b \cdot 232$, or returns "no solution" if no such a and b exist.

Remember to separately write your specification of recursive formula in plain English , your recursive solution for the formula and its proof of correctness , the algorithm using either memoization or bottom-up dynamic programming , and runtime analysis.

Solution.

```
def special(n):
  array products;
  products[0] = (0,0)

  for i in range(1, n + 1):
    if i >= 197 and products[i - 197] is not None:
        a, b = products[i - 197]
         products[i] = (a + 1, b)

      elif i >= 232 and products[i - 232] is not None:
          a, b = products[i - 232]
          products[i] = (a, b + 1)

  # Check if we found a valid combination for the target n
  if products[n] is not None:
      return products[n]
  else:
      return "no solution"
```

1. **Base:** The number zero can be formed with a =0 and b= 0.
2. **Inductive Step:** If any number is special, then subtracting 197 or 232 will give a smaller special number. Conversely, counting up by 197 and 232 will give us every subsequent special number. 0+197 = 197, 197 + 197 = 2(197), 197 + 232, etc.

This is going to run in logn times, since the max number of times it would take to confirm that a number isn't special is $log_{197}(n) * log_{232}(n)$

Problem 4. For the most part, we haven't been optimizing for space in our dynamic programs. But for many of the problems we've discussed it is possible to get away with much less space. The goal of this problem is for you do out one such example.

Recall the subsetsum problem from class. INPUT: set $S = \{s_1, \ldots, s_n\}$ of non-negative integers and a target number B. OUTPUT: set $S' \subseteq S$ with $\sum_{x \in S'} = B$ or FALSE if no such S' exists.

The algorithm we showed in class used $O(nB)$ time and $O(nB)$ space. Write pseudocode for an algorithm that solves subset sum in $O(nB)$ time but only needs $O(B)$ space in addition to the input itself. Your algorithm must return the actual set S'.

Solution. _____

Problem 5. ImportExport(A,B)

Input: Two arrays $A[0 \ldots n-1]$ and $B[0 \ldots n-1]$, both of length n, representing the prices of tulips in country A and country B at different times, respectively. All entries in both arrays are positive integers.

Output: Return the maximum possible profit, defined as:

$$\max_{j>i}\{B[j] - A[i]\}$$

or return 0 if all such $B[j] - A[i]$ are negative. You only need to return the value of the maximum profit, not the actual indices i and j.

Example:

Given the input:

$$A = [40, 18, 20, 25, 12, 13, 19, 20, 5, 7, 3]$$
$$B = [15, 50, 5, 30, 34, 19, 28, 33, 20, 20, 20]$$

The maximum possible profit is achieved by buying tulips at time $i = 4$ (when $A[4] = 12$) and selling them at time $j = 7$ (when $B[7] = 33$), which gives a profit of:

$$B[7] - A[4] = 33 - 12 = 21$$

Therefore, the output is 21.

- Give a recursive $O(n)$-time algorithm for ImportExport(A,B).
- Give a $O(n)$-time dynamic programming algorithm for ImportExport(A,B).

Solution. _____

Challenge Yourself. You are given a string consisting of just the characters $'a'$, $'b'$, or $'c'$. The task is to sort the string in lexicographical order ($a < b < c$) by finding the minimum number of swaps needed to achieve this. Each swap can exchange two characters in the string.